

Project Beacon — Comprehensive Test Matrix

Generated: 2026-08-17

Scope: Unit Testing · Integration Testing · End-to-End (E2E) Testing

Codebase: server/ (Node.js/Express) · enrichment_pipeline/ (Python/FastAPI) · drafting_service/ (Python/FastAPI) · client/ (React/TanStack)

Table of Contents

- 1. [Authentication, Sessions & Security](#)
- 2. [User & Team Management](#)
- 3. [Lead Management & Lifecycle](#)
- 4. [Deduplication & Identity Resolution \(Groq Pipeline\)](#)
- 5. [Automated Enrichment Pipeline](#)
- 6. [AI Message Drafting Service](#)
- 7. [Email Queue & Outreach Dispatch](#)
- 8. [Multi-Channel Conversations & LinkedIn Messaging \(Unipile\)](#)
- 9. [Unipile Account Connection & Webhook Processing](#)
- 10. [Client & Market Demand / Requirements](#)
- 11. [Google Sheets Sync](#)
- 12. [Recruiter Performance & KPI Rubric Scoring](#)
- 13. [Escalations & Alerts](#)
- 14. [Dashboard Aggregations & Headcount Sync](#)
- 15. [Background Jobs & Cron Scheduling](#)
- 16. [Frontend Component & UI Tests](#)
- 17. [Cross-Service E2E Journeys](#)
- 18. [Recommended Test Stack](#)

1. Authentication, Sessions & Security

Source files: `auth.routes.ts` , `auth.service.ts` , `middleware/auth.ts` , `middleware/rbac.ts` , `lib/normalize.ts`

Unit Tests

#	Test Case	File Under Test
1.1	<code>hashPassword</code> produces a bcrypt hash that <code>verifyPassword</code> confirms	<code>auth.service.ts</code>
1.2	<code>verifyPassword</code> rejects wrong password, empty password, and null hash	<code>auth.service.ts</code>
1.3	<code>verifyPasswordTimingSafe</code> pays bcrypt cost even when user is null (timing attack defense)	<code>auth.service.ts</code>
1.4	<code>generateAccessToken</code> produces a JWT with correct payload (id, email, name, role) and 15m expiry	<code>auth.service.ts</code>
1.5	<code>generateRefreshToken</code> creates a SHA-256-hashed token record with 7-day expiry	<code>auth.service.ts</code>
1.6	<code>verifyRefreshToken</code> rejects revoked, expired, and nonexistent tokens	<code>auth.service.ts</code>
1.7	<code>revokeRefreshToken</code> sets <code>revoked = true</code> on the matching hash	<code>auth.service.ts</code>
1.8	<code>normalizeEmail</code> strips zero-width characters, NFC-normalizes, trims, lowercases	<code>lib/normalize.ts</code>
1.9	<code>validateEmailFormat</code> rejects empty, >254 chars, missing @ or domain	<code>lib/normalize.ts</code>
1.10	<code>normalizeName</code> NFC-normalizes, strips invisible chars, collapses whitespace, caps at 80 chars	<code>lib/normalize.ts</code>
1.11	<code>requireRole</code> middleware returns 401 for missing user, 403 for disallowed role, calls <code>next()</code> for allowed role	<code>middleware/rbac.ts</code>

1.12	authenticateJwt middleware rejects missing Bearer token, expired token, and tampered signature	middleware/auth.ts
------	--	--------------------

Integration Tests

#	Test Case	Endpoint
1.13	Login with valid credentials returns accessToken + refreshToken cookie + user payload	POST /api/auth/login
1.14	Login with wrong password returns 401 with generic "Invalid email or password"	POST /api/auth/login
1.15	Login with nonexistent email returns 401 (same timing as wrong-password)	POST /api/auth/login
1.16	Login with deactivated account returns 403 ACCOUNT_DISABLED	POST /api/auth/login
1.17	Signup with valid fields creates user, returns 201, sends verification token	POST /api/auth/signup
1.18	Signup with duplicate email returns 409 USER_EXISTS	POST /api/auth/signup
1.19	Signup with invalid role returns 400 INVALID_ROLE	POST /api/auth/signup
1.20	Refresh with valid cookie returns new accessToken and rotates refreshToken	POST /api/auth/refresh
1.21	Refresh with revoked token returns 401	POST /api/auth/refresh
1.22	Logout revokes refresh token	POST /api/auth/logout
1.23	Email verification with valid token sets emailVerified=true and clears verifyToken	POST /api/auth/verify-email
1.24	Forgot-password generates resetToken with expiry	POST /api/auth/forgot-password
1.25	Reset-password with valid token updates passwordHash and clears token	POST /api/auth/reset-password
1.26	Reset-password with expired or wrong token returns 400	POST /api/auth/reset-password

E2E Tests

#	Test Case
1.27	Owner login → redirects to /owner dashboard
1.28	Recruiter login → redirects to /recruiter dashboard
1.29	Contractor login → redirects to /contractor/leads
1.30	Contractor navigating to /owner/clients → redirects to /unauthorized
1.31	Full password reset flow: forgot-password → email link → reset form → login with new password

2. User & Team Management

Source files: `user.routes.ts` , `auth.service.ts`

Unit Tests

#	Test Case	File Under Test
2.1	createUserSchema (Zod) rejects missing name, invalid email, and invalid role	user.routes.ts
2.2	Temp password generation produces URL-safe base64 string of sufficient entropy	user.routes.ts

Integration Tests

#	Test Case	Endpoint
---	-----------	----------

2.3	Owner lists recruiters: returns users with role=RECRUITER and SAFE_USER_SELECT fields (no passwordHash)	GET /api/users?role=RECRUITER
2.4	Owner lists contractors: returns users with managingRecruiterId from ContractorAssignment join	GET /api/users?role=CONTRACTOR
2.5	Recruiter listing users returns same data as owner (access allowed)	GET /api/users?role=RECRUITER
2.6	Contractor listing users returns 403	GET /api/users
2.7	Owner creates a new recruiter: returns 201 with user + tempPassword	POST /api/users
2.8	Creating user with duplicate email returns 409	POST /api/users
2.9	Non-owner creating user returns 403	POST /api/users
2.10	Owner soft-deletes user: sets isActive=false, returns updated user	DELETE /api/users/:id
2.11	Owner updates any user's languages	PATCH /api/users/:id/languages
2.12	Recruiter updates only their own languages; updating another user's languages returns 403	PATCH /api/users/:id/languages
2.13	Contractor assignment: recruiter assigns contractor to self via upsert	POST /api/users/:id/contractor-assignment
2.14	Owner assigns contractor to a specific recruiter via recruiterId body param	POST /api/users/:id/contractor-assignment
2.15	Contractor unassignment: removes ContractorAssignment row	DELETE /api/users/:id/contractor-assignment

E2E Tests

#	Test Case
2.16	Owner navigates to /owner/recruiters → sees full recruiter roster with KPI tiles
2.17	Owner adds new recruiter → recruiter appears in list → can login with temp password
2.18	Recruiter navigates to /recruiter/contractors → sees assigned/unassigned contractor lists

3. Lead Management & Lifecycle

Source files: `lead.routes.ts` , `lead.service.ts`

Unit Tests

#	Test Case	File Under Test
3.1	<code>buildLeadWhere</code> correctly combines q (text search), stage, language, country, service, recruiterId, flag, and since filters	<code>lead.routes.ts</code>
3.2	<code>buildLeadWhere</code> with no filters returns empty where clause	<code>lead.routes.ts</code>
3.3	<code>createLeadSchema</code> (Zod) rejects missing fullName, missing source, invalid email format, invalid LeadSource enum	<code>lead.routes.ts</code>
3.4	<code>claimLead</code> atomically sets claimedByRecruiterId only when currently null	<code>lead.service.ts</code>
3.5	<code>claimLead</code> throws 409 ALREADY_CLAIMED when lead already has a claimedByRecruiterId	<code>lead.service.ts</code>
3.6	<code>getLeadTimeline</code> merges and sorts stage_history, flag_events, interaction_events, and manual_activity_logs chronologically	<code>lead.service.ts</code>

Integration Tests

#	Test Case	Endpoint
3.7	Create lead: returns new lead with PENDING enrichment, unique maskedLabel, identityResolved=false	POST /api/leads
3.8	Create lead as contractor: sets createdByContractorId, does not set createdByRecruiterId	POST /api/leads
3.9	Create lead as recruiter: sets createdByRecruiterId and assignedRecruiterId to self	POST /api/leads
3.10	Create lead runs findDuplicateLead and sets dupFlagged/dupFlaggedField when match found	POST /api/leads
3.11	List leads (global): recruiter sees only identity-resolved+COMPLETE leads PLUS own assigned/created leads	GET /api/leads
3.12	List leads (global): owner sees all leads regardless of enrichment status	GET /api/leads
3.13	List leads with cursor-based pagination: returns correct page and nextCursor	GET /api/leads
3.14	List my leads (recruiter): returns only assigned or claimed leads	GET /api/leads/mine
3.15	List my leads (contractor): returns only createdByContractorId=self leads	GET /api/leads/mine
3.16	Get single lead: contractor can only view own submitted lead, else 403	GET /api/leads/:id
3.17	Get single lead with timeline: returns merged chronological activity events	GET /api/leads/:id
3.18	Patch lead: updating stage from NEW→CONTACTED creates StageHistory record	PATCH /api/leads/:id
3.19	Patch lead: moving to COLD without closureReason returns 400 REASON_REQUIRED	PATCH /api/leads/:id
3.20	Patch lead: setting enrichmentStatus=COMPLETE also sets identityResolved=true and strips ON_HOLD flag	PATCH /api/leads/:id
3.21	Patch lead: contractor can only edit their own submitted leads, else 403	PATCH /api/leads/:id
3.22	Patch lead: updating email/identityResolved auto-updates corresponding EmailQueueItem candidateName/subject	PATCH /api/leads/:id
3.23	Claim lead: first claim succeeds; concurrent second claim returns 409	POST /api/leads/:id/claim
3.24	Assign lead: sets assignedRecruiterId and assignedAt	POST /api/leads/:id/assign
3.25	Add flag: creates LeadFlagEvent record and adds to lead.flags array	POST /api/leads/:id/flags
3.26	Remove flag: creates LeadFlagEvent(action=REMOVED) and filters lead.flags	DELETE /api/leads/:id/flags/:flag
3.27	Log manual activity: creates ManualActivityLog with type=INTERVIEW or CALL	POST /api/leads/:id/activities
3.28	Batch delete: cascades across EmailQueueItem, ConversationMessage, Conversation, LeadFlagEvent, InteractionEvent, then Lead	POST /api/leads/batch-delete
3.29	CSV export: returns valid CSV with correct headers, respects active filter set, capped at 5000 rows	GET /api/leads/export
3.30	Check duplicate: exact email match returns isDuplicate=true with matchedField="email_address"	POST /api/leads/check-duplicate
3.31	Check duplicate: exact phone match returns isDuplicate=true with matchedField="contact_number"	POST /api/leads/check-duplicate
3.32	Check duplicate: exact fullName match (case-insensitive) returns isDuplicate=true	POST /api/leads/check-duplicate
3.33	Check duplicate: no match returns isDuplicate=false	POST /api/leads/check-duplicate

E2E Tests

#	Test Case
---	-----------

3.34	Recruiter adds lead via Add Lead dialog → lead appears in "My Leads" with PENDING enrichment badge
3.35	Contractor submits lead → lead appears in contractor's <code>/contractor/leads</code> list
3.36	Recruiter claims lead from Global Pool tab → lead moves to "My Leads" tab
3.37	Lead stage progression: NEW → CONTACTED → REPLIED → NEGOTIATING → INVITE_SENT → ONBOARDED via status dropdown
3.38	Moving lead to COLD prompts closure reason dialog → reason saved in StageHistory
3.39	Bulk CSV upload: 10 rows imported → toast shows success count → leads appear in list
3.40	Bulk select + delete: selected leads removed → dependent queue items and conversations cleaned up

4. Deduplication & Identity Resolution (Groq Pipeline)

Source files: `enrichment_pipeline/core/dedup.py` , `dedup_client.py` , `dedup_prompts.py` , `tests/test_dedup.py`

Unit Tests (Python)

#	Test Case	File Under Test
4.1	<code>_normalize_text</code> strips accents, casefolds, collapses whitespace ("Pérez " → "perez")	<code>dedup.py</code>
4.2	<code>_normalize_email</code> normalizes and returns empty string for empty/null values	<code>dedup.py</code>
4.3	<code>_normalize_phone</code> strips non-digit characters (" +44 7889 232438" → "447889232438")	<code>dedup.py</code>
4.4	<code>_name_tokens</code> extracts (first, last) from Full_Name, falls back to First_Name	<code>dedup.py</code>
4.5	<code>_name_similarity</code> handles token reordering ("John Smith" vs "Smith, John")	<code>dedup.py</code>
4.6	<code>_email_username_similarity</code> compares local parts only, ignoring domain	<code>dedup.py</code>
4.7	Step 1 — Exact email match (case-insensitive) flags duplicate with score=1.0 and 0 AI calls	<code>dedup.py</code>
4.8	Step 1 — Exact phone match (digit-normalized) flags duplicate with 0 AI calls	<code>dedup.py</code>
4.9	Step 2 — Blocking filter includes candidates with 4-char name prefix match	<code>dedup.py</code>
4.10	Step 2 — Blocking filter includes candidates sharing email domain	<code>dedup.py</code>
4.11	Step 2 — Unrelated names with different domains produce empty blocked list, no AI call	<code>dedup.py</code>
4.12	Step 2 — Blocked list is capped at BLOCKING_MAX_CANDIDATES=20	<code>dedup.py</code>
4.13	Step 3 — Narrowing sorts by max(name_similarity, email_username_similarity) and keeps top NARROWING_MAX_CANDIDATES=10	<code>dedup.py</code>
4.14	Step 4 — Groq client correctly parses JSON response <code>{"matches": [...]}</code>	<code>dedup_client.py</code>
4.15	Step 4 — Groq client retries on 429 and 5xx with exponential backoff	<code>dedup_client.py</code>
4.16	Step 4 — Groq client raises DedupGroqError after max_retries exhausted	<code>dedup_client.py</code>
4.17	Step 4 — Shortlist candidate_index is correctly mapped back to original batch index	<code>dedup.py</code>
4.18	Step 5 — Confidence < threshold (default 0.8) is discarded; ≥ threshold creates DuplicateCandidate	<code>dedup.py</code>
4.19	Missing GROQ_API_KEY gracefully skips AI stage, exact matches still flagged	<code>dedup.py</code>
4.20	Groq API failure for one lead does not abort dedup for remaining leads in batch	<code>dedup.py</code>
4.21	Malformed match entries (missing candidate_index, out-of-range index) are logged and skipped	<code>dedup.py</code>
4.22	<code>build_dedup_system_prompt</code> returns strict JSON-only system prompt	<code>dedup_prompts.py</code>
4.23	<code>build_dedup_user_content</code> formats tested lead + indexed candidates using CANONICAL_FIELDS	<code>dedup_prompts.py</code>

4.24	Single-lead batch (no prior leads) returns empty candidates list	dedup.py
------	--	----------

Integration Tests

#	Test Case	Endpoint
4.25	Batch enrichment with duplicates: POST /enrich/batch returns duplicate_review_queue with flagged pairs	Enrichment FastAPI
4.26	CLI mode: --input with duplicate leads writes output/duplicate_review_queue.json	CLI
4.27	_attach_duplicate_flags correctly decorates pipeline results with duplicate_flag.flagged=true for involved indices	main.py
4.28	Node POST /api/leads/check-duplicate returns exact-match results from Prisma	Node Server

E2E Tests

#	Test Case
4.29	User submits lead with email matching existing lead → toast warning "A similar lead may already exist"
4.30	Bulk CSV import containing 5 duplicate rows flags them without blocking the remaining 45 rows

5. Automated Enrichment Pipeline

Source files: enrichment_pipeline/ (orchestrator.py, parsers/, providers/, llm_fallback/*, core/field_audit.py, core/schema.py)

Server integration: server/src/jobs/enrichment.job.ts

Unit Tests (Python)

#	Test Case	File Under Test
5.1	linkedin_parser.py extracts name, headline, location, skills, experience from LinkedIn HTML snapshot	parsers/linkedin_parser.py
5.2	proz_parser.py extracts languages, services, years of experience, country from ProZ profile HTML	parsers/proz_parser.py
5.3	ada_parser.py parses Audio Description Association member profile fields	parsers/ada_parser.py
5.4	ata_parser.py parses American Translators Association directory entry	parsers/ata_parser.py
5.5	ataa_parser.py parses Audiovisual Translators Association directory entry	parsers/ataa_parser.py
5.6	bodalgo_parser.py parses Bodalgo voice-over artist profile	parsers/bodalgo_parser.py
5.7	generic_parser.py handles unknown source with best-effort extraction	parsers/generic_parser.py
5.8	source_router.py routes to correct parser based on Lead Source/Profile_Link domain	core/source_router.py
5.9	field_audit.py reports enrichment_complete only when all critical fields (Email, Contact, YOE) populated	core/field_audit.py
5.10	field_audit.py reports enrichment_partial with missing_critical_fields list when gaps exist	core/field_audit.py
5.11	schema.py — CANONICAL_FIELDS list matches the 13-field lead schema	core/schema.py
5.12	schema.py — is_empty_value correctly identifies None, empty string, and whitespace-only as empty	core/schema.py

5.13	<code>brightdata_client.py</code> constructs correct API request with <code>dataset_id</code> and API key	<code>providers/brightdata_client.py</code>
5.14	<code>brightdata_client.py</code> handles timeout, 429, and 5xx errors with retry	<code>providers/brightdata_client.py</code>
5.15	<code>tavily_client.py</code> constructs correct search request for profile URL	<code>providers/tavily_client.py</code>
5.16	<code>tavily_client.py</code> handles timeout and API errors gracefully	<code>providers/tavily_client.py</code>
5.17	LLM fallback client sends correct prompt when parser fails to extract critical fields	<code>llm_fallback/client.py</code>
5.18	LLM fallback response is parsed and merged without overwriting existing non-empty fields	<code>llm_fallback/client.py</code>
5.19	Orchestrator never overwrites existing populated fields with empty/null extraction results	<code>orchestrator.py</code>
5.20	Orchestrator executes pipeline stages in order: scrape → parse → LLM fallback → field audit	<code>orchestrator.py</code>

Integration Tests

#	Test Case	Endpoint
5.21	POST <code>/enrich</code> with LinkedIn lead: returns enriched lead JSON, correct <code>enrichment_status</code> , <code>execution_time_ms</code>	Enrichment FastAPI
5.22	POST <code>/enrich</code> with ProZ lead: returns enriched lead with parsed languages and YOE	Enrichment FastAPI
5.23	POST <code>/enrich/batch</code> with 5 leads: returns array of results with per-lead <code>field_sources</code>	Enrichment FastAPI
5.24	GET <code>/health</code> returns <code>{"status": "healthy"}</code>	Enrichment FastAPI
5.25	Node enrichment worker polls PENDING leads → marks IN_PROGRESS → calls <code>/enrich</code> → marks COMPLETE on success	<code>enrichment.job.ts</code>
5.26	Node enrichment worker on HTTP failure: reverts lead to PENDING for next poll cycle	<code>enrichment.job.ts</code>
5.27	Node enrichment worker on <code>enrichment_partial</code> response: sets FLAGGED_REVIEW, <code>identityResolved=false</code>	<code>enrichment.job.ts</code>
5.28	Node enrichment worker on <code>enrichment_complete</code> : sets <code>identityResolved=true</code> , <code>promotedToGlobalAt</code> , <code>justEnrichedUntil</code> (+24h)	<code>enrichment.job.ts</code>
5.29	Node enrichment worker correctly maps <code>displayName</code> from pipeline's <code>Full_Name/First_Name</code>	<code>enrichment.job.ts</code>

E2E Tests

#	Test Case
5.30	User adds lead with only LinkedIn URL → lead shows "Enriching..." → background worker completes → card shows real name, email, country
5.31	Partially enriched lead shows FLAGGED_REVIEW badge → recruiter manually enriches missing fields → lead promoted to Global pool

6. AI Message Drafting Service

Source files: `drafting_service/` (`main.py`, `draft_generator.py`, `evaluator.py`, `orchestrator.py`, `prompts/prompt_builder.py`, `claude_client.py`, `config.py`)

Server integration: `server/src/lib/messageTemplates.ts` , `email-queue.routes.ts` , `conversation.routes.ts`

Unit Tests

#	Test Case	File Under Test
6.1	<code>buildEmailDraft</code> template substitutes name and language correctly; uses "there" for missing name	<code>messageTemplates.ts</code>
6.2	<code>buildLinkedInDraft</code> produces a concise DM without subject line	<code>messageTemplates.ts</code>
6.3	<code>languageOf</code> returns <code>targetLanguage</code> first, fallback to <code>sourceLanguage</code> , null if both missing	<code>messageTemplates.ts</code>
6.4	<code>firstNameOf</code> prioritizes <code>displayName</code> > <code>firstName</code> > first token of <code>fullName</code> > "there"	<code>messageTemplates.ts</code>
6.5	<code>prompt_builder.py</code> — system prompt forbids hallucinating rates, credentials, or unverified skills	<code>prompts/prompt_builder.py</code>
6.6	<code>prompt_builder.py</code> — user prompt includes all 13 lead fields when present	<code>prompts/prompt_builder.py</code>
6.7	<code>evaluator.py</code> — lead eligibility check rejects leads missing target language or any contact channel	<code>evaluator.py</code>
6.8	<code>evaluator.py</code> — INELIGIBLE verdict returns descriptive flag list	<code>evaluator.py</code>
6.9	<code>draft_generator.py</code> — email channel includes both subject and body	<code>draft_generator.py</code>
6.10	<code>draft_generator.py</code> — linkedin channel produces shorter, subjectless body	<code>draft_generator.py</code>
6.11	<code>claude_client.py</code> — Groq/Claude API call constructs correct payload	<code>claude_client.py</code>
6.12	<code>claude_client.py</code> — handles timeout and API errors with retry	<code>claude_client.py</code>

Integration Tests

#	Test Case	Endpoint
6.13	POST <code>/draft</code> (email channel): returns personalized subject + body using lead attributes	Drafting FastAPI
6.14	POST <code>/draft</code> (linkedin channel): returns concise body without subject	Drafting FastAPI
6.15	POST <code>/draft</code> with ineligible lead: returns verdict=INELIGIBLE with flags	Drafting FastAPI
6.16	GET <code>/health</code> returns healthy status	Drafting FastAPI
6.17	POST <code>/api/email-queue/:id/generate-draft</code> : calls drafting service, persists generated subject+body, sets <code>aiGenerated=true</code>	Node Server
6.18	POST <code>/api/email-queue/:id/generate-draft</code> : INELIGIBLE lead returns 422 LEAD_NOT_DRAFT_ELIGIBLE	Node Server
6.19	POST <code>/api/email-queue/:id/generate-draft</code> : drafting service unreachable returns 502 DRAFTING_SERVICE_UNAVAILABLE	Node Server
6.20	POST <code>/api/conversations/:id/generate-draft</code> : returns draft body for LinkedIn compose box	Node Server

E2E Tests

#	Test Case
6.21	Recruiter opens email queue → clicks "Generate AI Draft" → editor populates with personalized message → recruiter edits and approves
6.22	Recruiter generates draft for lead missing email → 422 error shown with "add missing info" guidance

7. Email Queue & Outreach Dispatch

Source files: email-queue.routes.ts , outreach.routes.ts

Unit Tests

#	Test Case	File Under Test
7.1	Email queue send schema validates required body (non-empty), valid channel enum	email-queue.routes.ts
7.2	Batch send schema validates array of UUIDs, min=1, max=200	email-queue.routes.ts
7.3	toApiError normalizes UnipileService errors into ApiError shape	email-queue.routes.ts

Integration Tests

#	Test Case	Endpoint
7.4	Get email queue: returns only items where recruiterId=currentUser	GET /api/email-queue
7.5	Add lead to queue: creates EmailQueueItem with REVIEW_NEEDED status, template-generated subject/body	POST /api/email-queue
7.6	Add same lead again: returns existing item (idempotent)	POST /api/email-queue
7.7	Autosave: PATCH updates subject/body on matching owned item	PATCH /api/email-queue/:id
7.8	Autosave: PATCH on another recruiter's item returns 404 (no ownership leak)	PATCH /api/email-queue/:id
7.9	Send via LinkedIn: calls UnipileService.sendLinkedInMessage, marks SENT with sentAt and sentChannel=LINKEDIN	POST /api/email-queue/:id/send
7.10	Send via Email: calls UnipileService.sendEmail, marks SENT with sentChannel=EMAIL	POST /api/email-queue/:id/send
7.11	Send with missing LinkedIn profile: returns 400 MISSING_LINKEDIN_PROFILE	POST /api/email-queue/:id/send
7.12	Send with Unipile failure: rolls back, does not mark as SENT	POST /api/email-queue/:id/send
7.13	Batch send: best-effort per item, one failure does not abort remaining sends	POST /api/email-queue/batch-send
7.14	Batch send: items without any contact target return NO_CONTACT_TARGET error per-item	POST /api/email-queue/batch-send
7.15	Outreach send endpoint: dispatches via Unipile, records InteractionEvent, deletes EmailQueueItem if provided	POST /api/outreach/send

E2E Tests

#	Test Case
7.16	Recruiter adds lead to email queue from lead card → item appears in /recruiter/email-queue
7.17	Recruiter selects 3 queued items → clicks "Send All" → batch send executes → items transition to SENT
7.18	Recruiter edits draft subject/body → autosave persists → reload shows saved content

8. Multi-Channel Conversations & LinkedIn Messaging (Unipile)

Source files: conversation.routes.ts , services/unipile.service.ts

Unit Tests

#	Test Case	File Under Test
---	-----------	-----------------

8.1	<code>toApiError</code> normalizes raw Unipile errors (statusCode, code, message) into <code>ApiError</code>	<code>conversation.routes.ts</code>
8.2	<code>UnipileService.sendLinkedInMessage</code> constructs correct payload with <code>attendee_id</code> derived from profile URL	<code>unipile.service.ts</code>
8.3	<code>UnipileService.sendEmail</code> constructs correct email payload with <code>from/to/subject/body</code>	<code>unipile.service.ts</code>

Integration Tests

#	Test Case	Endpoint
8.4	BUG FIX VERIFICATION: GET <code>/api/conversations</code> with recruiter role returns ONLY conversations where <code>recruiterId=currentUser</code> (not all)	GET <code>/api/conversations</code>
8.5	GET <code>/api/conversations</code> with owner role returns all conversations	GET <code>/api/conversations</code>
8.6	GET <code>/api/conversations/:id</code> : recruiter can only view own thread; accessing another recruiter's thread returns 403	GET <code>/api/conversations/:id</code>
8.7	Create conversation: find-or-create by (<code>leadId</code> , <code>recruiterId</code> , <code>channel=LINKEDIN</code>)	POST <code>/api/conversations</code>
8.8	Create conversation with nonexistent <code>leadId</code> returns 404	POST <code>/api/conversations</code>
8.9	Send message: dispatches via Unipile, creates <code>ConversationMessage(sender=ME)</code> , updates <code>lastMessageAt</code>	POST <code>/api/conversations/:id/messages</code>
8.10	Send message on non-LINKEDIN channel returns 400 <code>UNSUPPORTED_CHANNEL</code>	POST <code>/api/conversations/:id/messages</code>
8.11	Send message with Unipile API failure: no DB record created, error surfaced to client	POST <code>/api/conversations/:id/messages</code>
8.12	Send message to lead without <code>profileLink</code> returns 400 <code>MISSING_LINKEDIN_PROFILE</code>	POST <code>/api/conversations/:id/messages</code>
8.13	Generate conversation draft: calls drafting service with <code>channel=linkedin</code>	POST <code>/api/conversations/:id/generate-draft</code>

E2E Tests

#	Test Case
8.14	Recruiter opens <code>/recruiter/conversations</code> → selects candidate → types and sends message
8.15	Inbound webhook delivers reply → unread badge appears on conversation → message displays in thread
8.16	Recruiter generates AI draft in conversation → draft populates compose box → recruiter sends

9. Unipile Account Connection & Webhook Processing

Source files: `unipile.routes.ts` , `services/unipile.service.ts`

Unit Tests

#	Test Case	File Under Test
9.1	<code>UnipileService.mintHostedAuthLink</code> generates valid nonce and stores <code>UnipileAuthAttempt</code>	<code>unipile.service.ts</code>
9.2	<code>UnipileService.handleWebhookEvent</code> validates webhook token and secret header	<code>unipile.service.ts</code>
9.3	Webhook deduplication: idempotent processing via <code>dedupeKey</code> in <code>UnipileWebhookEvent</code> table	<code>unipile.service.ts</code>
9.4	Account degradation tracking: status change from OK → <code>RECONNECTION_NEEDED</code> creates <code>AccountDegradation</code> row	<code>unipile.service.ts</code>

Integration Tests

#	Test Case	Endpoint
9.5	Connect account: mints hosted auth link, returns URL and nonce	POST /api/unipile/connect
9.6	Connect with missing provider returns 400	POST /api/unipile/connect
9.7	Reconnect account: mints reconnect-mode hosted auth link	POST /api/unipile/reconnect
9.8	List connected accounts for current user	GET /api/unipile/accounts
9.9	Disconnect account: removes ConnectedAccount and cascades AccountDegradation	DELETE /api/unipile/accounts/:accountId
9.10	Webhook: valid token + secret processes event and returns 200	POST /api/unipile/webhook/:token
9.11	Webhook: invalid token/secret returns 400 WEBHOOK_FAILED	POST /api/unipile/webhook/:token
9.12	Webhook: duplicate dedupeKey is silently ignored (idempotent)	POST /api/unipile/webhook/:token
9.13	Webhook account.status_changed: updates ConnectedAccount.status and creates degradation history	POST /api/unipile/webhook/:token
9.14	Webhook message.received: correlates unipileChatId → appends ConversationMessage(sender=THEM) → sets unread=true	POST /api/unipile/webhook/:token

E2E Tests

#	Test Case
9.15	Recruiter navigates to Settings → clicks "Connect LinkedIn" → completes OAuth → account appears with OK status
9.16	Account status degrades to RECONNECTION_NEEDED → warning badge appears on Settings page → recruiter reconnects

10. Client & Market Demand / Requirements

Source files: `client.routes.ts` , `client-demand.routes.ts` , `requirement.routes.ts`

Unit Tests

#	Test Case	File Under Test
10.1	<code>createClientSchema</code> (Zod) validates name (min 1, max 160), optional industry/contactName/contactEmail/notes	<code>client.routes.ts</code>
10.2	<code>createDemandSchema</code> validates required clientName, language, services array (min 1), valid priority enum	<code>client-demand.routes.ts</code>
10.3	<code>serviceSchema</code> validates service name (non-empty) and needed count (int, ≥0)	<code>client-demand.routes.ts</code>
10.4	Headcount calculation: sum of all service.needed values equals parent demand headcountNeeded	<code>client-demand.routes.ts</code>
10.5	<code>createRequirementsSchema</code> validates clientId (UUID), items array (min 1)	<code>requirement.routes.ts</code>
10.6	<code>requirementItemSchema</code> validates title, language, service, headcountNeeded, priority	<code>requirement.routes.ts</code>
10.7	<code>assignSchema</code> validates nullable recruiterId (UUID) and optional note	<code>requirement.routes.ts</code>
10.8	<code>patchRequirementSchema</code> validates optional datetime deadline and string notes	<code>requirement.routes.ts</code>

Integration Tests

#	Test Case	Endpoint
---	-----------	----------

10.9	List clients: returns all clients sorted by name (owner + recruiter access)	GET /api/clients
10.10	Create client: find-or-create by case-insensitive name; existing returns 200, new returns 201	POST /api/clients
10.11	Create demand: single transaction creates Client (if new) + ClientDemand + ClientDemandService rows + matching Requirement rows	POST /api/client-demands
10.12	Create demand: gap = headcountNeeded, filled = 0 on all created rows	POST /api/client-demands
10.13	Create demand: contractor role can read demands but cannot create (returns 403)	POST /api/client-demands
10.14	List demands: includes serviceBreakdown and client name, ordered by submittedAt desc	GET /api/client-demands
10.15	List requirements: filters by clientId, status, priority, and text search (q)	GET /api/requirements
10.16	Create requirements: bulk-create with client validation, auto-sets status=ACTIVE when recruiterId provided	POST /api/requirements
10.17	Create requirements: creates RequirementAssignment audit record when recruiterId provided at creation	POST /api/requirements
10.18	Patch requirement: updates deadline and notes only	PATCH /api/requirements/:id
10.19	Patch nonexistent requirement returns 404	PATCH /api/requirements/:id
10.20	Assign recruiter: updates recruiterId, transitions UNASSIGNED→ACTIVE, creates RequirementAssignment audit row	POST /api/requirements/:id/assign
10.21	Unassign recruiter: sets recruiterId=null, transitions to UNASSIGNED, logs audit event with recruiterId=null	POST /api/requirements/:id/assign
10.22	Assign nonexistent requirement returns 404	POST /api/requirements/:id/assign

E2E Tests

#	Test Case
10.23	Owner creates new client demand (e.g. "Acme Corp — Japanese Dubbing, needed: 3") → requirement appears in table with UNASSIGNED badge
10.24	Owner assigns Recruiter A to requirement → status changes to ACTIVE → Recruiter A sees it under "My Assigned" tab in /recruiter/clients
10.25	Owner unassigns recruiter → status reverts to UNASSIGNED
10.26	Recruiter filters requirements by language and priority → table updates correctly
10.27	Owner clicks requirement row → drill-down sheet opens showing covering leads for that language
10.28	Owner opens Recruiter–Language Mapping dialog → configures language assignments

11. Google Sheets Sync

Source files: `sheet-sync.routes.ts` , `google-sheets-sync-section.tsx`

Integration Tests

#	Test Case	Endpoint
11.1	Get sync config: returns sheetUrl and lastSyncedAt for current user (or defaults)	GET /api/sheet-sync
11.2	Put sync config: upserts sheet URL for current user	PUT /api/sheet-sync
11.3	Put invalid URL returns Zod validation error	PUT /api/sheet-sync

11.4	Trigger sync: currently returns {synced: false, reason: "not configured yet"}	POST /api/sheet-sync/sync
11.5	Contractor cannot access sheet sync endpoints (403)	All sheet-sync routes

E2E Tests

#	Test Case
11.6	Owner navigates to Settings → pastes Google Sheet URL → saves → sheet URL persisted
11.7	Owner clicks "Sync Now" → sees "not configured" message (until Google API integration is wired)

12. Recruiter Performance & KPI Rubric Scoring

Source files: `evaluation.routes.ts` , `jobs/scoring.job.ts`

Unit Tests

#	Test Case	File Under Test
12.1	<code>normalizedContribution</code> with <code>HIGHER_IS_BETTER</code> : ratio = $\min(1, \text{current/goodBand}) \times \text{weight}$	<code>scoring.job.ts</code>
12.2	<code>normalizedContribution</code> with <code>LOWER_IS_BETTER</code> : penalizes values exceeding <code>goodBand</code>	<code>scoring.job.ts</code>
12.3	<code>normalizedContribution</code> with <code>scored=false</code> or <code>weight=0</code> returns 0	<code>scoring.job.ts</code>
12.4	<code>normalizedContribution</code> with <code>current=0</code> returns 0	<code>scoring.job.ts</code>
12.5	Time-to-first-touch calculation: average days between lead creation and first outbound event	<code>scoring.job.ts</code>
12.6	Progression rate: % of stage transitions that move forward (not into COLD)	<code>scoring.job.ts</code>
12.7	Reason logged rate: % of COLD closures with non-null reason	<code>scoring.job.ts</code>
12.8	Onboard vs Queue: onboarded leads / total assigned leads × 100	<code>scoring.job.ts</code>
12.9	Overall score: sum of <code>normalizedContribution</code> across all 9 RUBRIC entries	<code>scoring.job.ts</code>
12.10	<code>latestPerMetricKey</code> returns first-seen (latest effectiveDate) row per unique <code>metricKey</code>	<code>evaluation.routes.ts</code>

Integration Tests

#	Test Case	Endpoint
12.11	Get KPI config: returns latest effectiveDate row per <code>metricKey</code>	GET /api/kpi-config
12.12	Patch KPI config (owner only): creates new versioned row with merged values, returns 201	PATCH /api/kpi-config/:metricKey
12.13	Patch KPI config: non-owner returns 403	PATCH /api/kpi-config/:metricKey
12.14	Patch KPI config: nonexistent <code>metricKey</code> returns 404	PATCH /api/kpi-config/:metricKey
12.15	Get recruiter score: returns latest <code>RecruiterScoreSnapshot</code> + <code>metricSnapshots</code>	GET /api/recruiters/:id/score
12.16	Get recruiter score for new recruiter (no snapshot): returns {snapshot: null, <code>metricSnapshots</code> : []}	GET /api/recruiters/:id/score
12.17	Get KPI summary: returns cached <code>RecruiterKpiSummary</code> for roster view	GET /api/recruiters/:id/kpi-summary
12.18	<code>computeRecruiterScoreSnapshot</code> creates/upserts <code>RecruiterScoreSnapshot</code> , <code>RecruiterMetricSnapshot</code> rows, and <code>RecruiterKpiSummary</code>	<code>scoring.job.ts</code>

12.19	ensureKpiConfigSeeded idempotently seeds missing KpiConfig rows from RUBRIC	scoring.job.ts
12.20	runMonthlyScoring processes all active recruiters (excludes owner/contractor)	scoring.job.ts

E2E Tests

#	Test Case
12.21	Owner navigates to /owner/recruiter-evaluation/:id → sees 5 metric group sections with trend arrows and normalized scores
12.22	Owner navigates to /owner/recruiters → roster displays overall score, outreach volume, and turnaround days per recruiter

13. Escalations & Alerts

Source files: `escalation.routes.ts` , `jobs/escalation.job.ts` , `recruiter-notifications-popover.tsx`

Unit Tests

#	Test Case	File Under Test
13.1	SLA breach detection: identifies inbound urgent replies with no recruiterRespondedAt older than 24h	escalation.job.ts
13.2	Stale lead detection: identifies leads with identityResolved=false or ON_HOLD flag older than 5 days	escalation.job.ts
13.3	Email queue backlog: triggers when recruiter's unsent items ≥ 25	escalation.job.ts
13.4	escalationExists prevents duplicate escalation creation for same category+lead	escalation.job.ts
13.5	Priority assignment: stale leads >10 days get P2; 5–10 days get P3	escalation.job.ts

Integration Tests

#	Test Case	Endpoint
13.6	scanForEscalations creates SLA Breach (P1), Recruiter Performance (P2/P3), and Email Queue Threshold Alert (P2) escalations	escalation.job.ts
13.7	Get escalations (owner): returns all, ordered by priority asc then createdAt desc	GET /api/escalations
13.8	Get escalations (recruiter): returns only own escalations	GET /api/escalations
13.9	Patch escalation status: owner can update any; recruiter can only update own (else 403)	PATCH /api/escalations/:id
13.10	Patch escalation: assignToMe sets ownerUserId to current user	PATCH /api/escalations/:id
13.11	Patch nonexistent escalation returns 404	PATCH /api/escalations/:id

E2E Tests

#	Test Case
13.12	Escalation fires → Bell icon shows unread badge count in navbar
13.13	Recruiter clicks notification popover → sees escalation cards with category, detail, timestamp
13.14	Recruiter clicks "Review Lead" action link → routes to the relevant lead
13.15	Recruiter clicks "Review email queue" action link → routes to /recruiter/email-queue

14. Dashboard Aggregations & Headcount Sync

Source files: `owner.index.tsx` , `recruiter.index.tsx` , `contractor.index.tsx` , `owner.pipelines.tsx`

Integration Tests

#	Test Case	Scope
14.1	Owner dashboard: total leads, leads by stage, leads by enrichment status aggregate correctly from real DB	Dashboard queries
14.2	Recruiter dashboard: "My Pipeline" metrics match leads assigned to current recruiter	Dashboard queries
14.3	Contractor dashboard: only counts leads where createdByContractorId=self	Dashboard queries
14.4	MISSING — TO IMPLEMENT: When lead stage moves to ONBOARDED/PLACED, matching Requirement.filled increments and gap decrements	Lead → Requirement sync
14.5	MISSING — TO IMPLEMENT: When Requirement.filled reaches headcountNeeded, status auto-transitions to FULFILLED	Requirement status sync
14.6	MISSING — TO IMPLEMENT: Parent ClientDemand.filled and ClientDemandService.filled update in same transaction	Demand sync

E2E Tests

#	Test Case
14.7	Owner dashboard shows correct counts by stage (pipeline bar chart)
14.8	Recruiter dashboard shows "My Leads" count, "Pending Enrichment" count, and "Conversations" count
14.9	Requirement table metrics tile shows correct Total / Unassigned / Active / Fulfilled counts

15. Background Jobs & Cron Scheduling

Source files: `jobs/index.ts`

Integration Tests

#	Test Case	Scope
15.1	<code>startBackgroundJobs</code> schedules 3 cron jobs: enrichment (*/*3min), escalations (hourly), scoring (monthly 1st 3am)	<code>jobs/index.ts</code>
15.2	Enrichment cron: calls <code>pollPendingEnrichment</code> which fetches up to 20 PENDING leads oldest-first	<code>enrichment.job.ts</code>
15.3	Enrichment cron: individual lead failure does not block processing of remaining batch	<code>enrichment.job.ts</code>
15.4	Escalation cron: calls <code>scanForEscalations</code> which runs SLA, stale, and backlog scans in parallel	<code>escalation.job.ts</code>
15.5	Scoring cron: calls <code>runMonthlyScoring</code> for all active recruiters; individual failure does not block rest	<code>scoring.job.ts</code>

16. Frontend Component & UI Tests

Source files: `client/src/components/features/*` , `client/src/routes/*` , `client/src/lib/*`

Component / Hook Tests (Vitest + React Testing Library)

#	Test Case	Component
16.1	AddLeadDialog: validates required fullName, optional email format, source selection	add-lead-dialog.tsx
16.2	AddLeadDialog: calls api.checkDuplicateLead before submitting; shows toast warning on match	add-lead-dialog.tsx
16.3	AddLeadDialog: CSV template download generates valid file	add-lead-dialog.tsx
16.4	ContractorAddLeadDialog: searches existing leads before submitting; enforces contractor field constraints	contractor-add-lead-dialog.tsx
16.5	ManualEnrichmentDialog: renders all enrichment fields; validates and submits patch	manual-enrichment-dialog.tsx
16.6	AssignRecruiterDialog: renders recruiter dropdown; submits assignment via api	assign-recruiter-dialog.tsx
16.7	ClientDemandDialog: multi-step form for client + services; validates headcount, priority	client-demand-dialog.tsx
16.8	RecruiterLanguageMappingDialog: renders language-recruiter mapping grid	recruiter-language-mapping-dialog.tsx
16.9	RecruiterNotificationsPopover: renders escalation items, marks all read on open	recruiter-notifications-popover.tsx
16.10	ConversationsPageView: renders conversation list, message thread, compose box	conversations-page-view.tsx
16.11	EmailQueuePageView: renders queue items, search/filter, send/batch-send buttons	email-queue-page-view.tsx
16.12	EvaluationDashboard: renders 5 metric group sections with trend arrows	evaluation-dashboard.tsx
16.13	EscalationsComponent: renders priority-sorted escalation cards	escalations.tsx
16.14	DateRangeToggle: emits correct date range on selection	date-range-toggle.tsx
16.15	LeadCard / RecruiterLeadCard: renders lead summary with stage badge, flags, priority pill	lead-card.tsx , recruiter-lead-card.tsx
16.16	SearchLeadDialog: searches and selects lead for adding to queue/conversation	search-lead-dialog.tsx
16.17	GoogleSheetsSyncSection: renders sheet URL input, sync button, last-synced timestamp	google-sheets-sync-section.tsx
16.18	ConnectAccountDialog / SelectAccountDialog: renders provider selection, handles Unipile auth flow	connect-account-dialog.tsx , select-account-dialog.tsx
16.19	ClientLogo: renders deterministic avatar based on client name	client-logo.tsx
16.20	PerformancePageView: renders recruiter/contractor performance metrics	performance-page-view.tsx
16.21	RoleGuard: renders children only for allowed roles; redirects otherwise	role-guard.tsx
16.22	Auth state (useAuth hook): persists user session, handles token refresh	lib/auth.ts
16.23	API client (api.ts): attaches Bearer token, handles 401 with token refresh retry	lib/api.ts

17. Cross-Service E2E Journeys

These tests verify the complete lifecycle across all three services (Node server, Python enrichment, Python drafting) and the React frontend.

#	Journey	Steps
17.1	Full Sourcing-to-Placement	Owner creates demand → assigns recruiter → recruiter imports leads via CSV → enrichment runs → recruiter claims from Global → sends AI-drafted outreach → candidate replies → interview logged → lead placed → demand fulfilled
17.2	Contractor Submission Flow	Contractor submits lead via form → duplicate check runs → lead created with PENDING → enrichment completes → lead promoted to Global → recruiter claims and progresses
17.3	AI Outreach Full Loop	Recruiter adds lead to email queue → generates AI draft → edits draft → sends via LinkedIn → Unipile delivers → inbound reply webhook → conversation thread updated → escalation fires if SLA breached
17.4	Recruiter Scoring Cycle	Recruiter performs outreach (creates InteractionEvents, progresses leads) → monthly cron runs → RecruiterScoreSnapshot computed → owner views evaluation dashboard with accurate metrics
17.5	Identity Resolution Pipeline	Batch import with ambiguous names ("Danny M", "Daniel Miller") → enrichment pipeline scrapes profiles → Groq dedup identifies same-person pair → duplicate_review_queue populated → future UI presents resolution options
17.6	Account Degradation & Recovery	Recruiter connects LinkedIn → account status OK → Unipile webhook signals RECONNECTION_NEEDED → account degradation logged → settings page shows warning → recruiter reconnects → status restored to OK
17.7	Market Demand Dashboard Sync	Owner creates demand for 3 German translators → recruiter sources and places 2 → dashboard shows 2/3 filled, 1 gap → recruiter places 3rd → requirement auto-transitions to FULFILLED

18. Recommended Test Stack

Layer	Framework / Tools	Scope
Node.js Backend Unit & Integration	Vitest + Supertest + prisma-mock (or test DB with migrations)	Express routes, RBAC, DB transactions, scoring rubric, services
Python Enrichment Pipeline	pytest + pytest-mock + httpx	Parsers, dedup stages, Groq client, BrightData/Tavily clients, orchestrator
Python Drafting Service	pytest + pytest-mock + httpx	Prompt builder, eligibility evaluator, Claude/Groq client, draft generator
Frontend Components & Hooks	Vitest + @testing-library/react + MSW (mock service worker)	Dialog forms, table filters, notification popover, auth state, API client
End-to-End (E2E)	Playwright (recommended) or Cypress	Full browser journeys across roles, cross-service data flows

Summary Statistics

Category	Unit Tests	Integration Tests	E2E Tests	Total
1. Auth & Security	12	14	5	31
2. User & Team Mgmt	2	13	3	18
3. Lead Management	6	27	7	40
4. Dedup & Identity Resolution	24	4	2	30
5. Enrichment Pipeline	20	9	2	31
6. AI Drafting Service	12	8	2	22
7. Email Queue & Outreach	3	12	3	18
8. Conversations & LinkedIn	3	10	3	16

9. Unipile Accounts & Webhooks	4	10	2	16
10. Client & Market Demand	8	14	6	28
11. Google Sheets Sync	0	5	2	7
12. KPI Rubric Scoring	10	10	2	22
13. Escalations & Alerts	5	6	4	15
14. Dashboard Aggregations	0	6	3	9
15. Background Jobs	0	5	0	5
16. Frontend Components	23	0	0	23
17. Cross-Service Journeys	0	0	7	7
TOTALS	132	153	53	338

Document generated for Project Beacon test planning. Each numbered test case corresponds to a specific, implementable test that can be tracked in a test management system.